

Finite-Domain Quantified Nullivance Logic: A Machine-Checked Completeness Theorem for a Macro-Free Extensional Tableau*

Trinh Tung Lam[†]

July 2026 — draft v0.1

Abstract

We extend nullivance propositional logic (NPL) — a paraconsistent, two-channel logic on the unit square with a harmonization connective $\oplus$ and four threshold signs T^+, T^-, F^+, F^- — by finite-domain quantifiers. The language is function-free first-order with crisp equality; models interpret predicates over a fixed finite nonempty domain by values in the four-element matrix **4**, and the finite universal (existential) quantifier takes the pointwise minimum (maximum) on the truth channel and the dual on the falsity channel. Main results, each machine-checked in Lean 4: (i) the exact threshold projection of the propositional theory lifts to the quantified language over finite domains; (ii) a natural signed tableau calculus is *refuted* as complete — crisp equality $x=x$ is valid but underivable — and two further tempting proof routes are refuted by explicit countermodels, all machine-checked; (iii) the repaired *core extensional tableau*, whose closure conditions add four crisp-equality clauses and two ground-closure clauses to the decomposition rules, is sound and *complete*: a branch closes exactly when no finite model satisfies it. The completeness proof is a quantified tableau engine driven by a domain-weighted measure ($\text{qsize}(\forall x \varphi) = (n+1) \cdot \text{qsize}(\varphi) + 1$ over a domain of size $n+1$) under which every rule strictly decreases branch weight, together with a canonical-model construction at the literal stage keyed by an injective ground-atom encoding. As a corollary, closure of

*Draft v0.1 (July 2026). Companion to *Nullivance Propositional Logic: Threshold-Signed Consequence on the Unit Square with a Machine-Checked Completeness Theorem* (same repository, `papers/npl-core`). Results marked [L] are verified sorry-free in the Lean 4 artifact described in Appendix A. The related-work section is a stub pending the project’s literature pass; novelty wording is correspondingly guarded.

[†]Contact: `lamyofficialvn@gmail.com`.

the grounded propositional branch under rigid constraints reaches the core calculus with no propositional-simulation macro — settling the artifact’s final open bridge conjecture. We also document the failure analysis that led from a derivation-by-derivation simulation program to the weighted-measure proof.

1 Introduction

The companion paper develops the propositional core of nullivance logic (NPL): formulas valued in $[0, 1]^2$ with independent truth and falsity channels, a threshold $\tau \in (0, 1]$ inducing four signs, an exact projection onto the four-valued matrix **4**, and a four-signed analytic tableau calculus with a machine-checked completeness theorem. This paper asks the next question: what does a *quantified* NPL look like, and can its metatheory be machine-checked to the same standard?

We deliberately restrict to **finite nonempty domains**. The restriction is not cosmetic: the exact-projection theorem — the load-bearing bridge between the continuous semantics and the finite matrix — uses that finite infima and suprema are attained, and it can fail over infinite domains when a supremum is approached but not attained (Remark 3.6). Finite domains keep the entire two-layer architecture of NPL intact, and they are the natural first-order setting for the intended applications of a threshold logic (finite data, finite scenarios). Equality is *crisp*: $x=y$ evaluates to **T** or **F** by identity of domain elements, never to a glut or a gap.

Contributions. All results are verified sorry-free in Lean 4 unless explicitly marked otherwise; [L] markers name the verifying declarations.

1. *Finite-domain quantified NPL* (Section 3): function-free syntax with predicate atoms, crisp equality, the NPL connectives including $\oplus$, and quantifiers evaluated by finite channel-wise extrema; quantifier duality under channel-swap negation; and the exact threshold projection for the quantified language.
2. *A catalogue of machine-checked refutations* (Section 5): the naive signed tableau is incomplete (crisp reflexivity is valid but underivable); the bare grounding bridge to propositional NPL is unsound as a completeness route; and a natural dispatcher shortcut for the completeness proof fails on a two-element counterexample. Each failure shaped the final calculus.

3. *The core extensional tableau and its exact semantic characterization* (Sections 6 and 7): the headline theorem states that a finite branch closes in the macro-free core calculus *iff* it has no finite model, hence derivability coincides with finite semantic consequence. The proof introduces a domain-weighted formula measure under which every rule of the calculus — including the $(n+1)$ -ary quantifier rules — strictly decreases the branch weight, and closes the literal stage by a canonical model read off the branch through an injective ground-atom encoding.
4. *The constrained-grounding corollary* (Section 8): closure of the grounded propositional branch augmented by *rigid constraints* (fixing the identity and equality atoms) yields core closure. Unlike the first completeness theorem of the artifact, no propositional-simulation macro rule appears anywhere: the propositional derivation is consumed through its soundness and the core closure is rebuilt semantically.
5. *A documented dead end* (Section 9): the derivation-by-derivation simulation program (replay traces with residual fold tails) is developed to the point where its exact obstruction is isolated — the binary propositional decomposition cascade cannot be reassembled into the core’s $(n+1)$ -ary quantifier rules — and the weighted-measure proof is shown to dissolve the obstruction. The simulation layer remains machine-verified as a study of the approach.

Method. The development follows the project’s rigor rules: every claim is labeled, refutation is attempted before proof, and the Lean artifact is the ground truth. The manuscript numbering is presentational; the stable numbering and the full proofs live in the repository documentation.

2 Preliminaries: propositional NPL

We summarize what is needed from the companion paper. The propositional language has atoms and $\neg, \wedge, \vee, \oplus$. The matrix **4** is $\{0, 1\}^2$ with corners **T** = (1, 0), **F** = (0, 1), **B** = (1, 1), **N** = (0, 0); negation swaps the two bits, $\wedge$ acts as (min, max), $\vee$ as (max, min), and $\oplus$ as (min, min) (bilattice consensus [5, 4, 1]). The $\{\neg, \wedge, \vee\}$ -fragment is Belnap–Dunn first-degree entailment [2, 3]. Four signs read the bits: T^+ (t -bit is 1), T^- (t -bit is 0), F^+ , F^- likewise on the f -bit. A four-signed analytic tableau calculus (sixteen decomposition rules, two closure conditions) is sound and complete for signed consequence over **4**, and — by the exact projection — over the

continuous semantics on $[0, 1]^2$; signed tableaux over sets-of-values signs are a standard paradigm [6]. All of this is machine-checked in the same artifact this paper extends.

3 Finite-domain quantified NPL

Definition 3.1 (Syntax). Fix countable stocks of variables and predicate symbols. Quantified formulas are generated by

$$\varphi ::= P(x_0, \dots, x_k) \mid x = y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \oplus \varphi \mid \forall x \varphi \mid \exists x \varphi.$$

Predicate atoms carry a list of variables; arity is a well-formedness discipline of the intended signature, and evaluation is total on the raw syntax. [L:FiniteF0.QFormula]

Definition 3.2 (Models and assignments). For a fixed $n \in \mathbb{N}$, the domain is $\text{Fin}(n+1)$ — finite and nonempty by construction. A *model* M assigns to each predicate symbol and argument list a value in **4**. An *assignment* ρ maps variables to domain elements; $\rho[x \mapsto d]$ updates it. [L:FiniteF0.QModel, FiniteF0.Assignment]

Definition 3.3 (Evaluation). Evaluation extends the propositional **4** clauses by crisp equality and finite channel-wise quantifiers:

$$V(x=y) = \begin{cases} \mathbf{T} & \rho(x) = \rho(y) \\ \mathbf{F} & \text{otherwise,} \end{cases} \quad V(\forall x \varphi) = \left(\bigwedge_d t_d, \bigvee_d f_d \right), \quad V(\exists x \varphi) = \left(\bigvee_d t_d, \bigwedge_d f_d \right),$$

where (t_d, f_d) is the value of φ under $\rho[x \mapsto d]$ and the extrema range over the finite domain. Signed satisfaction reads one bit of the value, exactly as in the propositional case. A *signed branch member* is a triple (sign, assignment, formula); a model satisfies a branch iff it satisfies every member. [L:FiniteF0.qeval, forallV4, existsV4, qsatSigned, qsatBranch]

Proposition 3.4 (Quantifier duality). $\neg\forall x \varphi$ and $\exists x \neg\varphi$ have the same value, as do $\neg\exists x \varphi$ and $\forall x \neg\varphi$: channel-swap negation exchanges the two finite extrema. The nonempty domain is used; over an empty domain the statement would depend on conventions for vacuous quantifiers. [L:FiniteF0.qeval_neg_all, qeval_neg_ex]

Theorem 3.5 (Finite exact projection). Interpret predicates by continuous truth-objects in $[0, 1]^2$ and evaluate quantifiers by finite channel-wise infima/suprema. For every threshold $0 < \tau \leq 1$, thresholding commutes

with evaluation: $\pi_\tau(V_M(\varphi)) = V_{\pi_\tau(M)}(\varphi)$, where $\pi_\tau(M)$ projects the predicate interpretations pointwise. Hence the finite-domain metatheory, like the propositional one, can be developed on **4** and lifted. `[L:FiniteFO.finite_exact_projection]`

Remark 3.6 (Why finite domains). Over an infinite domain the existential/supremum direction of Theorem 3.5 can fail when the supremum is not attained: every instance can fall below the threshold while the supremum reaches it. The finite, nonempty domain is therefore a load-bearing design choice, not a convenience. (Recorded as a refutation attempt in the development.)

4 Grounding

The finite domain makes every quantifier a finite fold, which suggests translating quantified branches into propositional ones.

Definition 4.1 (Ground atoms and the ground translation). *Ground atoms* are: two fixed identity atoms $\top_g$ and $\perp_g$ (fold units), a ground predicate atom for each predicate symbol and argument list, and a ground equality atom for each pair of domain elements. Ground atoms are encoded injectively into propositional atoms. The translation gr_ρ maps a quantified formula under an assignment to a propositional formula: predicate and equality atoms to their ground atoms; connectives homomorphically; $\forall x \varphi$ to the finite conjunction fold of $\{\text{gr}_{\rho[x \mapsto d]}(\varphi)\}_d$ (with unit $\top_g$) and $\exists$ to the disjunction fold (with unit $\perp_g$). `[L:FiniteFO.GroundAtom, groundAtomCode, FiniteFO.ground, foldConj, foldDisj]`

Lemma 4.2 (Grounding truth lemma). *For every model M there is a propositional valuation v_M (reading ground atoms through M , with $\top_g \mapsto \mathbf{T}$, $\perp_g \mapsto \mathbf{F}$, and equality atoms crisp) such that the propositional value of $\text{gr}_\rho(\varphi)$ under v_M equals the quantified value of φ under M, ρ . `[L:FiniteFO.ground_truth, groundVal]`*

Definition 4.3 (Rigid constraints). The *rigid constraints* form the finite propositional branch containing T^+, F^- for $\top_g$; T^-, F^+ for $\perp_g$; and for every pair (a, b) of domain elements, T^+, F^- (respectively T^-, F^+) for the ground equality atom of (a, b) when $a = b$ (respectively $a \neq b$). They pin exactly the part of a ground valuation that finite-domain models do not choose freely; ground predicate atoms remain unconstrained. `[L:FiniteFO.rigidGroundConstraints]`

5 Calculi, and what fails

The propositional tableau rules lift to signed quantified branch members, with four finite quantifier rules per quantifier: the *block* rules add all $n+1$ instances to the branch at once (non-branching: $T^+\forall$, $F^-\forall$, $T^-\exists$, $F^+\exists$), and the *per-element* rules require one closed child per domain element (the remaining four signs). Closure, in the naive calculus, is an identical opposite-signed pair. [L:FiniteFO.QCloses]

Proposition 5.1 (The naive calculus is incomplete). *Crisp reflexivity $T^+(x=x)$ is valid over every finite domain, but the empty branch does not derive it: no decomposition rule applies to an equality atom and no closure pair exists. Machine-checked countermodel to the completeness conjecture for QCloses. [L:FiniteFO.qeqRefl0_valid, qeqRefl0_not_derivable, qcompleteness_current_refuted]*

Definition 5.2 (Equality-completed calculus). Add four crisp-equality closure clauses: a branch closes if it contains $T^-(x=y)$ or $F^+(x=y)$ under an assignment with $\rho(x) = \rho(y)$, or $T^+(x=y)$ or $F^-(x=y)$ with $\rho(x) \neq \rho(y)$. [L:FiniteFO.QClosesEq]

Example 5.3 (Why equality clauses are not enough). Let $\rho(x) = \sigma(y) = d$ and consider the two-member branch $\{T^+(\rho, Px), T^-(\sigma, Py)\}$. Every model gives both members the same ground predicate value at $(P, [d])$, so the branch is unsatisfiable; but the members are not an identical pair (different formulas, different assignments), no rule decomposes literals, and no equality clause applies. Closure must be allowed on *equal groundings*, not merely on identical members.

6 The core extensional tableau

Definition 6.1 (Core calculus). The *core extensional finite-domain tableau* extends the equality-completed calculus of Definition 5.2 by two *ground closure clauses*: a branch closes if it contains members $T^+(\rho, \varphi)$ and $T^-(\sigma, \psi)$ with $\text{gr}_\rho(\varphi) = \text{gr}_\sigma(\psi)$, or F^+/F^- likewise. All sixteen propositional decomposition rules and all eight finite quantifier rules are retained. The calculus contains *no* propositional simulation macro. Derivability: $\Gamma \vdash_{\text{core}} s\varphi$ iff the branch $\{\bar{s}\varphi\} \cup \Gamma$ closes, where $\bar{s}$ is the opposite sign. [L:FiniteFO.QClosesExtCore, QDerivesExtCore]

Proposition 6.2 (Soundness). *If a branch closes in the core calculus, no finite model satisfies it; hence $\Gamma \vdash_{\text{core}} s\varphi$ implies $\Gamma \models_{Q,n} s\varphi$.
[L:FiniteFO.QClosesExtCore.unsat]*

7 The completeness theorem

Theorem 7.1 (Semantic completeness of the core calculus). *Let B be a finite branch of signed quantified members over domain size $n+1$. If no model with domain $\text{Fin}(n+1)$ satisfies B , then B closes in the core calculus. With Proposition 6.2, closure is exactly finite unsatisfiability, and*

$$\Gamma \vdash_{\text{core}} s\varphi \iff \Gamma \models_{Q,n} s\varphi.$$

[L:FiniteFO.QClosesExtCore.complete_of_unsat, qclosesExtCore_iff_unsat, QDerivesExtCore.complete, qDerivesExtCore_iff_qconsequence4]

Proof (engine; full details in the artifact). Step 1: the domain-weighted measure. Define qsize on quantified formulas by $\text{qsize} = 1$ on predicate and equality atoms, $+1$ for negation, $+1$ over the sum for binary connectives, and

$$\text{qsize}(\forall x \varphi) = \text{qsize}(\exists x \varphi) = (n+1) \cdot \text{qsize}(\varphi) + 1,$$

extended to branches by summation. Every rule of Definition 6.1 strictly decreases the weight of the undecomposed segment: unary and binary rules drop at least the head’s outermost symbol; a block quantifier rule replaces weight $(n+1)w + 1$ by the $n+1$ instances of weight w each; a per-element rule replaces it by a single instance of weight $w \leq (n+1)w$. The case analysis is exhaustive over the eight formula constructors and four signs.

Step 2: the engine. By strong induction on the weight bound, a branch split as *todo* + *lits* (with *lits* literals) that no finite model satisfies closes in the core: a literal head moves to *lits*; a compound head is decomposed by its rule, unsatisfiability transfers to each child by the corresponding local satisfaction equivalence (the signed quantifier equivalences and the propositional local-soundness table), the induction hypothesis applies at strictly smaller weight, and monotonicity of closure restores the full branch. Because the core’s quantifier rules are natively $(n+1)$ -ary, no intermediate “partially decomposed fold” state ever arises (contrast Section 9).

Step 3: the literal stage. For a branch of literals: if an equality literal contradicts its crisp value, an equality clause closes; if two literals carry T^+/T^- (or F^+/F^-) with equal groundings, a ground closure clause closes. Otherwise build the *canonical model*: the truth (falsity) bit of each ground

predicate atom is set iff some T^+ - (F^+ -) literal grounds to it, read through the injective ground-atom encoding. Positive literals are satisfied by their own witness; negative literals because the opposite witness would be a ground closure pair, excluded by assumption; equality literals by crispness, their four unsatisfiable sign/value combinations being exactly the equality clauses. This contradicts unsatisfiability, so a closure option applied. $\square$

Remark 7.2 (What the refutations bought). Three hypotheses of Theorem 7.1 carry machine-checked failure witnesses for their removal: without the ground closure clauses, Example 5.3 is unsatisfiable but open; without the $(n+1)$ -weighting, decomposing a block quantifier *increases* the unweighted measure and the induction has no footing; without any one of the four equality clauses, the corresponding literal configuration is unsatisfiable but unmodelable and unclosable.

8 Constrained grounding reaches the core

The artifact’s first finite-domain completeness theorem ran through the propositional calculus: if the grounded branch together with the rigid constraints closes propositionally, an *extensional macro calculus* closes the quantified branch by a single simulation rule. That macro rule is exactly what a proof-theoretic reading wants to eliminate.

Theorem 8.1 (Constrained grounding, macro-free). *For every finite branch B : if the propositional branch $\text{Rigid}_n + \text{gr}(B)$ closes in the propositional tableau calculus, then B closes in the core calculus of Definition 6.1. No simulation macro occurs: the propositional derivation is consumed through propositional soundness, and the core closure is rebuilt by Theorem 7.1. Concretely: a finite model of B would induce, by Lemma 4.2 and the rigid values, a propositional valuation satisfying $\text{Rigid}_n + \text{gr}(B)$, contradicting propositional soundness; hence B is finitely unsatisfiable and Theorem 7.1 applies. [L:FiniteFO.groundBranch_closes_to_core, satBranch_groundVal_rigid]*

Conversely, grounding is sound: the grounded branch of a satisfiable quantified branch is satisfiable under the induced valuation (Lemma 4.2), so Theorem 8.1 composes with the propositional completeness theorem of the companion paper into a round-trip between the two levels.

9 The simulation program: a documented dead end

An earlier phase of the project attempted Theorem 8.1 by *simulation*: induction on the propositional closure derivation, maintaining a replay trace whose items include quantified members, rigid atoms, and *residual fold tails* — the partially decomposed remains of a grounded quantifier, which are not groundings of any single quantified formula. The layer produced a complete, machine-verified dispatch analysis for propositional closure pairs (sixteen source combinations per closure sign), a recursive “tail consumer” closing any quantified formula against a fold tail it grounds into, and two further machine-checked refutations of shortcut routes.

The program then hit a precise wall: when the propositional derivation decomposes the grounding of a *per-element-sign* quantifier ($T^{-}\forall$ and its mirrors), the binary conjunction cascade produces a right child asserting the pooled fold of the remaining instances in one branch, while the core rule needs one closed child per domain element; the pooled induction hypothesis cannot be redistributed, and depth-preserving inversion does not make the induction well-founded. The weighted-measure engine of Theorem 7.1 dissolves the problem at the root: the core’s quantifier rules are $(n+1)$ -ary, so the binary cascade never appears. We record the dead end because the failure analysis, not luck, produced the measure; the simulation layer remains in the artifact as a verified study.

10 Related work

This section is a stub pending the project’s systematic literature pass; wording is deliberately guarded. Signed tableaux with sets-of-values signs for finitely-valued logics, including quantified ones, are a standard paradigm [6]; quantified Belnap–Dunn logics and their semantics are classical territory [2, 3, 7]. The NPL-specific ingredients here are the threshold-signed presentation with the harmonization connective in the object language, the crisp-equality and ground-closure clause set of the core calculus, the domain-weighted completeness engine, and the machine-checked status of the entire development including its refutations. Which of these ingredients appear elsewhere — in particular in the literature on tableaux for finitely-valued first-order logics and on formalized completeness theorems [8] — is exactly what the pending literature pass must establish before any novelty claim is made.

11 Open problems

(1) *Infinite domains*: the projection argument fails at unattained suprema (Remark 3.6); a quantified NPL over infinite domains needs either attained-extrema semantics or a genuinely different bridge. (2) *Function symbols and non-crisp equality*: the present language is function-free and its equality never takes glut or gap values; both restrictions are design choices worth revisiting. (3) *Complexity*: finite-domain consequence is decidable by finiteness of the model space; its exact complexity as a function of domain size and formula size is open here. (4) *The trace-calculus bridge*: the optional sharpening that the replay-trace calculus itself absorbs constrained propositional closure remains a conjecture in the repository (no longer on any critical path).

A The Lean artifact

The results of this paper are verified in the **Nullivance** Lean 4 library (module `Nullivance.FiniteF0`, ≈ 380 declarations), part of the same artifact as the companion paper. **Toolchain.** Lean 4 (`leanprover/lean4:v4.32.0-rc1`) with `mathlib` pinned via the Lake manifest; `lake exe cache get && lake build` (approx. 2000 jobs). **Coverage policy.** The build is `sorry-free`; every result above is stated against the calculus `QClosesExtCore` directly. Axiom audit: every named declaration depends only on `propext`, `Classical.choice`, `Quot.sound`. **Key declarations.** Semantics: `qeval`, `qsatBranch`; projection: `finite_exact_projection`; refutations: `qcompleteness_current_refuted`, `qeqRef10_not_derivable`; calculus: `QClosesExtCore`; soundness: `QClosesExtCore.unsat`; completeness engine: `qsize`, `qweightB`, `qclosesCore_todo`, `qclosesCore_lits`; headline: `QClosesExtCore.complete_of_unsat`, `qDerivesExtCore_iff_qconsequence4`; grounding: `ground_truth`, `groundBranch_closes_to_core`.

References

- [1] Ofer Arieli and Arnon Avron. Reasoning with logical bilattices. *Journal of Logic, Language and Information*, 5(1):25–63, 1996.
- [2] Jr. Belnap, Nuel D. A useful four-valued logic. In J. Michael Dunn and George Epstein, editors, *Modern Uses of Multiple-Valued Logic*, pages 5–37. D. Reidel, Dordrecht, 1977. Page range per CrossRef record of the chapter DOI.

- [3] J. Michael Dunn. Intuitive semantics for first-degree entailments and “coupled trees”. *Philosophical Studies*, 29(3):149–168, 1976.
- [4] Melvin Fitting. Bilattices and the semantics of logic programming. *Journal of Logic Programming*, 11(2):91–116, 1991.
- [5] Matthew L. Ginsberg. Multivalued logics: A uniform approach to reasoning in artificial intelligence. *Computational Intelligence*, 4(3):265–316, 1988.
- [6] Reiner Hähnle. *Automated Deduction in Multiple-Valued Logics*, volume 10 of *International Series of Monographs on Computer Science*. Oxford University Press, Oxford, 1994. Sets-as-signs signed tableaux for finitely-valued logics.
- [7] Graham Priest. *An Introduction to Non-Classical Logic: From If to Is*. Cambridge University Press, Cambridge, 2 edition, 2008. Ch. 8: FDE, incl. signed tableaux with nodes $A, + / A, -$.
- [8] Jørgen Villadsen and Anders Schlichtkrull. Formalizing a paraconsistent logic in the Isabelle proof assistant. In *Transactions on Large-Scale Data- and Knowledge-Centered Systems XXXIV*, Lecture Notes in Computer Science, pages 92–122. Springer, 2017.